StoryPoints AI Internship Program

Week 3 Assignment — Big Data Processing with Spark & Data Warehousing in BigQuery

Learning Goals

- Understand why Spark is used for large-scale ETL
- Run Spark jobs on GCP Dataproc (cluster or serverless)
- Transform large datasets in parallel
- Load results into BigQuery partitioned/clustered tables
- Learn incremental loading, error handling, query optimization in a warehouse

Scenario

In Week 1 you built ETL scripts, and in Week 2 you orchestrated them in Cloud Composer. Now your manager wants you to process larger datasets using Spark on GCP Dataproc and load them into BigQuery. You will handle big data transformations, error handling, and optimize queries in the data warehouse.

Tasks

1. Setup Spark on GCP Dataproc (cluster or serverless). Test small jobs locally with PySpark.

2. Process Clickstream with Spark: parse timestamps, clean invalids, deduplicate by session_id, write Parquet partitioned by date(click_time).

3. Process Transactions with Spark: join with exchange rates, enrich amount_in_usd, partition by date(txn_time), cluster by user_id, write Parquet.

4. Load into BigQuery: create datasets raw_spark_outputs and curated_dw, load partitioned Parquet, maintain error tables for invalid rows.

5. Data Warehouse Queries: daily active users by country, revenue per currency vs USD (last 30 days), funnel from visits → cart → transactions, compare costs with/without partition filters.

Deliverables

Spark jobs (/week3_spark_jobs/)

GCS output Parquet files (partitioned)

BigQuery DDL scripts (schemas/)

Error handling strategy (error_table.sql)

Analytical queries (queries/week3_analysis.sql)

Updated README.md with Spark setup, data partitioning, BigQuery schema and queries

Evaluation Criteria

Aspect	Weight
Spark job correctness & partitioned output	30%
BigQuery schema design (partitioning & clustering)	25%
Error handling mechanism	15%
Incremental load logic	15%
Query optimization & documentation	15%

Pro Tips

Use Spark DataFrames (not RDDs) for optimizations.

Write outputs in Parquet (efficient for BigQuery).

Always partition on event time (click_time, txn_time).

Cluster BigQuery tables on high-cardinality fields like user_id.

Use bq load --autodetect or schema files for ingestion.

Use BigQuery dry-run/EXPLAIN to check query cost.